Bab 11

Analisis dan Perancangan Aplikasi

Memahami apa yang perlu dibangun sebelum menulis kode.

11

DI BAB INI

- Analisis kebutuhan
- Identifikasi object
- Class, attribute, dan method
- Class relationship
- Class diagram sederhana

Di bab-bab sebelumnya kita sudah menulis class, menjaga data dengan encapsulation, memakai inheritance dan polymorphism, menyimpan data ke file, serta membuat tampilan GUI. Sekarang pertanyaannya berubah: jika mendapat permintaan membuat sebuah aplikasi, bagaimana kita menentukan class apa yang perlu dibuat dan bagaimana class-class itu bekerja sama?

Bab ini membahas langkah dari kebutuhan pengguna menuju rancangan aplikasi. Studi kasusnya adalah **Sistem Peminjaman Perpustakaan**, yang mengembangkan contoh perpustakaan pada Bab 6, 7, dan 9. Rancangan di bab ini menjadi acuan implementasi pada Bab 12 dan pembuatan GUI pada Bab 13. Hasil akhirnya berupa kebutuhan yang dapat diperiksa, daftar object dan class, pembagian tanggung jawab, serta diagram yang saling sesuai.

Pendekatan dari spesifikasi menuju rancangan class juga digunakan dalam proyek terpadu oleh [North, 2025]. Hubungan antara benda dalam masalah, data, perilaku, inheritance, dan composition dibahas oleh [Lutz, 2025], khususnya pada Bab 31. Contoh dan keputusan rancangan dalam bab ini disusun untuk kebutuhan perpustakaan sederhana pada modul ini.

CAPAIAN PEMBELAJARAN

Setelah mempelajari bab ini, Anda diharapkan mampu:

- menuliskan kebutuhan aplikasi beserta aturan dan kriteria penerimaannya;
- membedakan pengguna aplikasi, object, class, attribute, dan method;
- menentukan tanggung jawab class serta alasan pemilihan relasinya;
- menggambar class diagram sederhana dengan multiplicity yang tepat;
- memeriksa kesesuaian rancangan melalui skenario berhasil dan gagal.

Bekal yang digunakan adalah materi class dan object, encapsulation, relationship, exception handling, file JSON, serta pemisahan model dan tampilan dari bab sebelumnya.

11.1 Analisis Kebutuhan Aplikasi

Analisis kebutuhan adalah kegiatan memahami masalah, siapa yang menggunakan aplikasi, pekerjaan yang perlu dibantu, dan batas keberhasilan aplikasi. Hasilnya harus cukup jelas untuk dibaca pengguna maupun programmer. Urutan memahami kebutuhan

sebelum merancang class juga ditekankan dalam bagian pengembangan framework pada [Anonim, tt].

Permintaan seperti “buat aplikasi perpustakaan yang lengkap” belum cukup untuk dijadikan rancangan. Kata “lengkap” dapat berarti berbeda bagi petugas, mahasiswa, dan programmer. Permintaan itu perlu diuraikan menjadi pekerjaan yang dapat diamati, misalnya mendaftarkan buku, mencatat peminjaman, dan melihat buku yang belum dikembalikan.

Masalah, Pengguna, dan Batas Aplikasi

Bayangkan sebuah ruang baca kampus mencatat peminjaman di buku tulis. Petugas sulit memastikan apakah sebuah buku sedang dipinjam, siapa peminjamnya, dan kapan harus dikembalikan. Ketika catatan diperbarui di beberapa tempat, status buku kadang berbeda dengan catatan peminjamannya.

Pada proyek ini, **petugas perpustakaan** menjadi pengguna aplikasi. Anggota menyampaikan permintaan kepada petugas; anggota belum mengoperasikan aplikasi sendiri. Petugas merupakan *aktor*, yaitu pihak yang berinteraksi dengan sistem. Seorang aktor tidak otomatis harus menjadi class di dalam model aplikasi.

RUANG LINGKUP PROYEK

- Aplikasi dipakai oleh satu petugas pada satu komputer dalam satu waktu.
- Data yang dikelola adalah buku, anggota, dan catatan peminjaman.
- Satu kode buku mewakili satu eksemplar buku cetak atau satu lisensi peminjaman e-book. Satu kode hanya dapat dipinjam oleh satu anggota pada waktu yang sama.
- Buku cetak dipinjam selama 14 hari, sedangkan e-book selama 7 hari. Angka ini merupakan aturan studi kasus, bukan ketentuan semua perpustakaan.
- Data disimpan dalam file JSON lokal. Tampilan desktop dibuat pada Bab 13.
- Versi awal belum mencakup login, pembayaran denda, pemesanan buku, penghapusan riwayat, maupun penggunaan bersama melalui jaringan.

Batas tersebut membantu menentukan pekerjaan yang akan diselesaikan pada Bab 12 dan 13. Fitur tambahan dapat dikembangkan setelah alur utama berjalan dan datanya konsisten.

Kebutuhan Fungsional

Kebutuhan fungsional menyatakan pekerjaan yang harus dapat dilakukan aplikasi. Berikan kode pada setiap kebutuhan agar mudah ditelusuri ke rancangan dan pemeriksaannya.

Perhatikan bahwa F03 menyebut keadaan yang harus diperiksa. Menyediakan tombol Pinjam saja belum memenuhi kebutuhan jika tombol tersebut masih mengizinkan dua

Tabel 11.1. Kebutuhan fungsional Sistem Peminjaman Perpustakaan.

Kode	Kebutuhan	Hasil yang diharapkan
F01	Mendaftarkan buku	Kode, judul, dan jenis buku tercatat; kode yang sama tidak didaftarkan dua kali.
F02	Mendaftarkan anggota	ID dan nama anggota tercatat; ID yang sama tidak didaftarkan dua kali.
F03	Meminjamkan buku	Catatan peminjaman dibuat jika anggota dan buku terdaftar serta buku tersedia.
F04	Mencatat pengembalian	Catatan peminjaman ditandai selesai tanpa menghapus riwayatnya.
F05	Melihat data dan status	Petugas dapat melihat daftar buku, ketersediaannya, dan peminjaman yang masih aktif.
F06	Menyimpan dan memuat	Buku, anggota, serta riwayat peminjaman tersedia kembali setelah aplikasi dibuka ulang.

anggota meminjam kode buku yang sama secara bersamaan.

Kebutuhan Nonfungsional

Kebutuhan nonfungsional menyatakan kualitas atau batas kerja aplikasi. Rumuskan dengan cara yang dapat diperiksa. Pernyataan "aplikasi harus mudah digunakan" perlu dilengkapi perilaku yang menunjukkan kemudahan tersebut.

Tabel 11.2. Kebutuhan nonfungsional dan cara memeriksanya.

Kode	Kualitas	Cara pemeriksaan
NF01	Konsistensi data	Peminjaman yang ditolak tidak menambah riwayat atau mengubah status buku.
NF02	Pesan yang membantu	Masukan tidak valid menghasilkan penjelasan penyebab dan data yang perlu diperbaiki.
NF03	Pemisahan tanggung jawab	Model dapat diperiksa dari terminal tanpa mengimpor atau membuka GUI.
NF04	Penyimpanan yang dapat diperiksa	JSON memakai UTF-8; file rusak dilaporkan dan tidak diam-diam dianggap sebagai data kosong.

Pemisahan komponen membantu kita memahami dan menguji aplikasi ketika ukurannya bertambah. Hal ini sejalan dengan penjelasan [Wilson, 2024] tentang membatasi interaksi antarkomponen agar rancangan tetap dapat dipahami.

Aturan Bisnis

Aturan bisnis menentukan keadaan yang sah dalam masalah yang sedang dimodelkan. Aturan ini harus dijaga oleh model atau layanan aplikasi, sehingga tetap berlaku ketika tampilan diganti.

1. Kode buku, ID anggota, dan ID peminjaman harus unik dalam kelompok datanya masing-masing. Kode dan nama wajib terisi.
2. Peminjaman hanya boleh dilakukan untuk anggota dan buku yang sudah terdaftar.
3. Sebuah kode buku boleh memiliki banyak riwayat peminjaman, tetapi paling banyak satu peminjaman aktif pada suatu waktu.
4. Setiap catatan peminjaman menghubungkan tepat satu anggota dan satu buku.
5. Tanggal jatuh tempo ditetapkan saat peminjaman dibuat, berdasarkan tanggal pinjam dan lama pinjam jenis buku tersebut.
6. Tanggal pengembalian tidak boleh mendahului tanggal pinjam. Peminjaman yang sudah selesai tidak dapat dikembalikan untuk kedua kalinya.
7. Pengembalian mempertahankan riwayat. Data yang gagal diproses tidak boleh meninggalkan perubahan sebagian.

Aturan ketiga dapat diperiksa dengan menghitung peminjaman aktif untuk sebuah kode buku. Kita tidak perlu menyimpan flag `tersedia` lain yang harus terus diselaraskan dengan riwayat peminjaman.

Menulis Skenario Penggunaan

Skenario menjelaskan satu pekerjaan dari sudut pandang pengguna. Tuliskan kondisi awal, langkah utama, hasil akhir, dan kemungkinan penolakan.

Skenario Meminjamkan Buku

1. **Kondisi awal:** anggota M001 dan buku B001 sudah terdaftar; B001 belum memiliki peminjaman aktif.
2. Petugas memilih anggota M001 dan buku B001, lalu memasukkan tanggal pinjam 27 Oktober 2026.
3. Aplikasi memeriksa keberadaan anggota, keberadaan buku, dan ketersediaannya.
4. Aplikasi meminta lama pinjam dari object buku. Karena B001 adalah buku cetak, lama pinjamnya 14 hari.
5. Aplikasi membuat catatan P001 dengan tanggal jatuh tempo 10 November 2026.
6. Petugas melihat peminjaman berhasil dan status B001 menjadi sedang dipinjam.
7. **Alur penolakan:** jika B001 masih dipinjam, aplikasi memberi pesan penolakan dan tidak

menambah catatan baru.

Skenario ini belum menentukan warna tombol atau letak tabel GUI. Yang harus disepakati lebih dahulu adalah data, aturan, dan hasil pekerjaannya. Bentuk antarmuka mengikuti alur tersebut pada Bab 13.

Kriteria Penerimaan dan Ketertelusuran

Kriteria penerimaan menjelaskan bukti bahwa suatu kebutuhan sudah terpenuhi. Hubungan kebutuhan, method, dan skenario pemeriksaan disebut *ketertelusuran* (*traceability*). Dengan hubungan ini, kita dapat menunjukkan alasan sebuah method diperlukan.

Tabel 11.3. Contoh penelusuran kebutuhan ke rancangan dan pemeriksaan.

Kode	Operasi yang dirancang	Kriteria penerimaan
F01	`tambah_buku(buku)`	Pendaftaran ulang B001 ditolak; buku yang sudah tercatat tidak tertimpa.
F03	`pinjamkan(...)`	Pinjam pertama berhasil; permintaan kedua untuk B001 yang masih aktif ditolak.
F04	`kembalikan(...)`	P001 selesai, B001 tersedia lagi, dan catatan P001 tetap ada.
F05	`tersedia(kode)`	Hasilnya sesuai dengan keberadaan peminjaman aktif untuk kode tersebut.
F06	`simpan()` dan `muat()`	Setelah dimuat ulang, identitas, tanggal, dan hubungan peminjaman tetap sama.

HASIL ANALISIS KEBUTUHAN

Sebelum melanjutkan, pastikan Anda dapat menjawab siapa pengguna aplikasi, data apa yang dikelola, operasi apa yang disediakan, aturan apa yang harus dijaga, dan bagaimana keberhasilannya diperiksa. Kebutuhan yang belum jelas perlu diperjelas sebelum diterjemahkan menjadi class.

11.2 Identifikasi Object

Object adalah satu entitas tertentu dalam model aplikasi, lengkap dengan identitas, keadaan, dan perilakunya. Mulailah dengan contoh konkret agar rancangan tidak berhenti pada nama-nama class yang masih abstrak.

Dalam sistem perpustakaan, buku berkode B001 adalah satu object, anggota M001 adalah object lain, dan catatan P001 yang menghubungkan keduanya juga dapat dimod-

elkan sebagai object. Tiga object tersebut mewakili peran yang berbeda walaupun terlibat dalam satu pekerjaan.

Menyaring Kata Benda dari Kebutuhan

Analisis kata benda dan kata kerja dari Bab 1 dapat dipakai kembali. Namun, hasilnya adalah *kandidat*, bukan daftar class yang langsung diterima seluruhnya. Lutz menjelaskan pemetaan benda dan tindakan sebagai langkah awal rancangan, lalu menunjukkan bagaimana object bekerja sama melalui relasi [Lutz, 2025].

Tabel 11.4. Penyaringan kandidat dari deskripsi aplikasi.

Kandidat	Keputusan	Alasan
Buku	Object domain	Memiliki kode, judul, dan aturan lama pinjam.
Anggota	Object domain	Memiliki identitas dan menjadi pihak yang meminjam.
Peminjaman	Object domain	Menyimpan hubungan anggota–buku beserta tanggal dan statusnya.
Judul dan tanggal	Attribute	Menjelaskan keadaan suatu entitas; belum memerlukan tanggung jawab class tersendiri.
Petugas	Aktor	Mengoperasikan aplikasi; versi ini belum mengelola data akun petugas.
JSON dan tombol	Format dan komponen tampilan	JSON adalah bentuk penyimpanan, sedangkan tombol dibahas saat merancang GUI.

Keputusan tersebut bergantung pada kebutuhan. Jika pada pengembangan berikutnya perpustakaan harus mengelola pergantian petugas dan hak akses, data petugas mungkin memerlukan class. Pada ruang lingkup sekarang, penambahan itu belum dibutuhkan.

Identitas dan Keadaan Object

Identitas membedakan satu entitas dari entitas lain. Dua eksemplar dengan judul sama tetap merupakan buku yang berbeda jika kodenya berbeda. Sebaliknya, buku B001 tetap buku yang sama setelah judulnya diperbaiki atau setelah dipinjam berkali-kali.

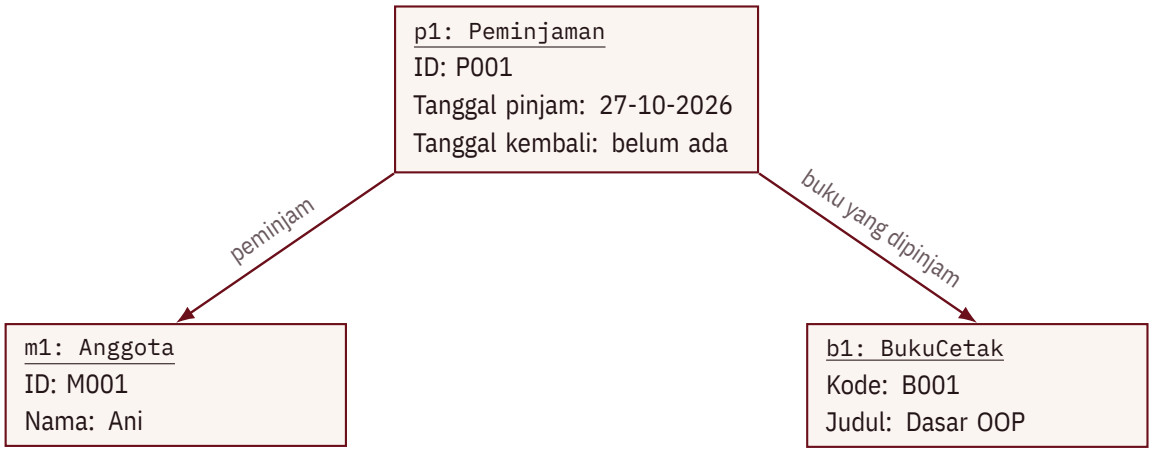
ID seperti M001 dan B001 merupakan identitas dalam aplikasi, bukan hasil fungsi `id()` Python. Nilai `id()` berkaitan dengan object selama program berjalan dan tidak digunakan sebagai identitas tetap dalam file JSON.

Tabel 11.5. Contoh object yang digunakan untuk memeriksa rancangan.

Object	Calon class	Keadaan contoh
B001	`BukuCetak`	Judul “Dasar OOP”, lama pinjam 14 hari.
B002	`BukuCetak`	Judul “Dasar OOP”, eksemplar lain yang dapat dipinjam secara terpisah.
E001	`EBook`	Judul “Dasar Python”, lama pinjam 7 hari.
M001	`Anggota`	Nama Ani; menjadi peminjam pada catatan P001.
P001	`Peminjaman`	Menghubungkan M001 dengan B001, dipinjam 27 Oktober 2026, belum dikembalikan.

Melihat Hubungan antar-Object

Gambar 11.1 memperlihatkan satu kejadian peminjaman. Diagram object menggambarkan contoh keadaan pada suatu saat, sedangkan class diagram nanti menggambarkan struktur yang dapat menghasilkan banyak object seperti itu.



Gambar 11.1. Satu object peminjaman merujuk satu anggota dan satu buku. Nama p1, m1, dan b1 adalah penanda object pada diagram.

Catatan P001 tidak perlu menyalin seluruh data anggota dan buku. Ia dapat menyimpan rujukan ke object yang sudah terdaftar. Jika ada dua catatan untuk M001, keduanya merujuk anggota yang sama. Hal ini perlu dipertahankan ketika data dimuat kembali dari file.

PERIKSA DENGAN DUA CONTOH

Untuk setiap calon class, coba tuliskan dua object yang mungkin dibuat darinya. Jika yang Anda tuliskan sebenarnya hanya dua nilai seperti “Ani” dan “Budi”, periksa apakah keduanya cukup menjadi nilai attribute `nama`. Object mahasiswa atau anggota mempunyai identitas dan peran yang lebih lengkap daripada nama saja.

11.3 Identifikasi Class, Attribute, dan Method

Setelah contoh object ditemukan, kita mengelompokkan object yang memiliki struktur dan perilaku sejenis ke dalam class. Untuk setiap class, tentukan **tanggung jawab**, **data yang perlu diketahui**, dan **operasi yang disediakan**. Pertanyaan ini membantu mencegah satu class mengerjakan semua urusan aplikasi.

Membagi Tanggung Jawab Class

Rancangan inti menggunakan enam class domain: `Buku`, `BukuCetak`, `EBook`, `Anggota`, `Peminjaman`, dan `Perpustakaan`. Dua class pendukung, yaitu kontrak `Penyimpanan` dan implementasi `PenyimpananJSON`, menangani batas antara model dan file.

Tabel 11.6. Tanggung jawab class domain.

Class	Tanggung jawab	Batas tanggung jawab
`Buku`	Menyimpan identitas buku dan mendefinisikan operasi `lama_pinjam()`.	Tidak memilih peminjam atau membuka file.
`BukuCetak`, `EBook`	Menentukan lama pinjam sesuai jenis buku.	Tidak mengelola seluruh riwayat perpustakaan.
`Anggota`	Menyimpan identitas anggota yang terdaftar.	Tidak mengubah ketersediaan buku secara langsung.
`Peminjaman`	Menyimpan satu transaksi dan mencatat penyelesaiannya.	Tidak mengatur tombol atau kotak pesan.
`Perpustakaan`	Mendaftarkan entitas, memeriksa aturan lintas-object, dan mengoordinasikan transaksi.	Tidak mengatur widget atau membaca format JSON sendiri.

Tabel 11.7. Tanggung jawab komponen penyimpanan.

Komponen	Tanggung jawab
`Penyimpanan`	Kontrak dua operasi: `muat()` mengembalikan dictionary data aplikasi dan `simpan(data)` menyimpan dictionary tersebut. Kontrak dapat dinyatakan dengan `Protocol` dari Bab 5.
`PenyimpananJSON`	Membaca serta menulis JSON pada path yang diberikan, dan melaporkan kesalahan file. Komponen ini tidak memutuskan apakah sebuah buku boleh dipinjam.

Pemisahan tersebut melanjutkan pola penyimpanan pada Bab 9. Payload proyek ini berupa dictionary yang menampung beberapa kelompok data, sehingga bentuknya lebih

lengkap daripada list buku pada contoh sebelumnya. Nama operasi tetap `muat()` dan `simpan(data)`.

Menentukan Attribute dan Jenis Datanya

Pilih attribute yang diperlukan untuk menjalankan aturan dan mempertahankan keadaan aplikasi. Tipe data membantu menjelaskan nilai yang diharapkan, meskipun validasinya tetap harus ditulis saat implementasi.

- **Buku**: `kode: str` dan `judul: str`. Kode bersifat tetap setelah pendaftaran. Class turunan menentukan lama pinjam melalui method, sehingga tidak perlu menyalin percabangan jenis buku ke seluruh aplikasi.
- **Anggota**: `id_anggota: str` dan `nama: str`. ID bersifat tetap; nama harus berisi teks setelah spasi berlebih dihapus.
- **Peminjaman**: `id_pinjam: str`, rujukan `anggota` dan `buku`, serta `tanggal_pinjam`, `tanggal_jatuh_tempo`, dan `tanggal_kembali`. Dalam model, tanggal memakai `date`; tanggal kembali bernilai `None` selama peminjaman aktif.
- **Perpustakaan**: dictionary buku berdasarkan kode, dictionary anggota berdasarkan ID, koleksi catatan peminjaman, object penyimpanan, dan penomoran ID peminjaman berikutnya.

Dictionary memudahkan pencarian buku B001 dan anggota M001 berdasarkan identitasnya. Koleksi peminjaman mempertahankan transaksi yang masih aktif maupun yang sudah selesai. Bentuk koleksi ini dapat dipilih kembali ketika implementasi, selama aturan dan antarmuka yang disepakati tetap terpenuhi.

Data Tersimpan, Data Turunan, dan Catatan Historis

Tidak semua informasi perlu disimpan sebagai attribute yang dapat diubah. Periksa dari mana sebuah nilai berasal dan apakah nilai itu harus mengikuti keadaan terkini atau mempertahankan keputusan pada masa lalu.

Tabel 11.8. Pemilihan data tersimpan dan data turunan.

Informasi	Keputusan	Alasan
Judul buku	Disimpan	Merupakan data dasar yang tidak dapat dihitung dari catatan peminjaman.
Status aktif peminjaman	Diturunkan	Aktif jika `tanggal_kembali` bernilai `None`; tidak perlu flag kedua.
Ketersediaan buku	Diturunkan	Tersedia jika tidak ada peminjaman aktif yang merujuk kode buku tersebut.
Tanggal jatuh tempo	Disimpan saat transaksi dibuat	Mempertahankan kesepakatan awal meskipun aturan lama pinjam berubah kemudian.

Untuk P001, tanggal jatuh tempo dihitung sekali saat peminjaman dibuat: 27 Oktober ditambah 14 hari menjadi 10 November 2026. Jika kebijakan buku cetak kelak berubah menjadi 10 hari, transaksi P001 tetap menggunakan tanggal yang sudah disepakati. Ini berbeda dari property terhitung yang selalu mengikuti nilai terbaru.

Menentukan Method dan Kontraknya

Method dipilih dari tindakan yang diperlukan oleh kebutuhan aplikasi. Sebuah *kontrak method* menjelaskan masukan, keadaan yang harus terpenuhi, hasil, dan cara melaporkan kegagalan. Kontrak membantu programmer model dan programmer GUI menggunakan pengertian yang sama.

Tabel 11.9. Operasi utama pada class `Perpustakaan`.

Method	Hasil	Aturan yang dijaga
`tambah_buku(buku)`	`None`	Kode valid dan belum terdaftar.
`tambah_anggota(anggota)`	`None`	ID valid dan belum terdaftar.
`pinjamkan(id_anggota, kode_buku, tanggal)`	Object `Peminjaman`	Anggota dan buku terdaftar; tidak ada peminjaman aktif untuk buku yang sama.
`kembalikan(id_pinjam, tanggal)`	`None`	Catatan ditemukan, masih aktif, dan tanggal kembali tidak mendahului tanggal pinjam.
`tersedia(kode_buku)`	`bool`	Kode buku terdaftar; hasil dihitung dari catatan aktif.

Tiga method pendukung menyediakan daftar untuk tampilan: `daftar_buku()`, `daftar_anggota()`, dan `daftar_peminjaman()`. Method terakhir menerima parameter opsional `aktif_saja=False`; jika diberi nilai `True`, hasilnya hanya memuat catatan aktif. Method `simpan()` dan `muat()` mengoordinasikan pemindahan keadaan aplikasi melalui object penyimpanan. Koleksi internal tidak diberikan sebagai jalur untuk mengubah data tanpa pemeriksaan; hasil daftar dapat berupa salinan koleksi.

Untuk `pinjamkan()`, urutan rancangannya adalah mencari anggota, mencari buku, memeriksa ketersediaan, meminta lama pinjam dari buku, membuat ID dan catatan baru, lalu memasukkannya ke koleksi. Pemeriksaan dilakukan sebelum koleksi diubah. Jika salah satu syarat gagal, keadaan perpustakaan tetap seperti semula.

LETAKKAN ATURAN PADA PEMILIKNYA

`Peminjaman` menjaga aturan tentang tanggal dan penyelesaian satu transaksi. `Perpustakaan` menjaga aturan yang memerlukan beberapa object, seperti keberadaan anggota dan ketersedi-

aan buku. GUI meminta operasi tersebut dan menampilkan hasil atau pesan exception; GUI tidak menjadi satu-satunya tempat pemeriksaan aturan.

Kesalahan dapat dilaporkan dengan exception yang sudah dipelajari di Bab 8. Contohnya `BukuSedangDipinjamError` untuk buku yang belum tersedia dan `DataRusakError` untuk data simpanan yang tidak dapat dipakai. Nama serta hierarki exception ditetapkan ketika implementasi; penyebab kegagalannya sudah dicatat dalam kontrak ini.

Merancang Bentuk Data Simpanan

Di dalam memori, `Peminjaman` merujuk object anggota dan buku. Di dalam JSON, hubungan itu disimpan melalui ID. Contoh berikut adalah rancangan isi file, bukan keluaran program yang sudah diimplementasikan.

Contoh rancangan data/perpustakaan.json

```
{
  "versi": 1,
  "nomor_pinjam_berikutnya": 2,
  "buku": [
    {"kode": "B001", "judul": "Dasar OOP", "jenis": "cetak"},
    {"kode": "E001", "judul": "Dasar Python", "jenis": "ebook"}
  ],
  "anggota": [
    {"id_anggota": "M001", "nama": "Ani"}
  ],
  "peminjaman": [
    {
      "id_pinjam": "P001",
      "id_anggota": "M001",
      "kode_buku": "B001",
      "tanggal_pinjam": "2026-10-27",
      "tanggal_jatuh_tempo": "2026-11-10",
      "tanggal_kembali": null
    }
  ]
}
```

Field `jenis` menentukan class buku yang dibuat saat pemuatan. Tanggal disimpan sebagai teks berformat ISO, lalu dikembalikan menjadi object `date` dalam model. Penomoran berikutnya ikut dipertahankan agar aplikasi tidak menggunakan kembali ID transaksi yang sudah tercatat.

Pemuatan dilakukan dengan membuat seluruh object buku dan anggota terlebih dahulu, kemudian menghubungkan catatan peminjaman melalui kode dan ID. Jika P001 merujuk M001 yang tidak ditemukan, data tidak boleh dianggap valid. Data juga perlu diperiksa terhadap kode ganda, tanggal yang tidak sah, dan dua peminjaman aktif untuk

buku yang sama. Pemeriksaan ini menjaga aturan yang sama seperti saat pengguna memasukkan data.

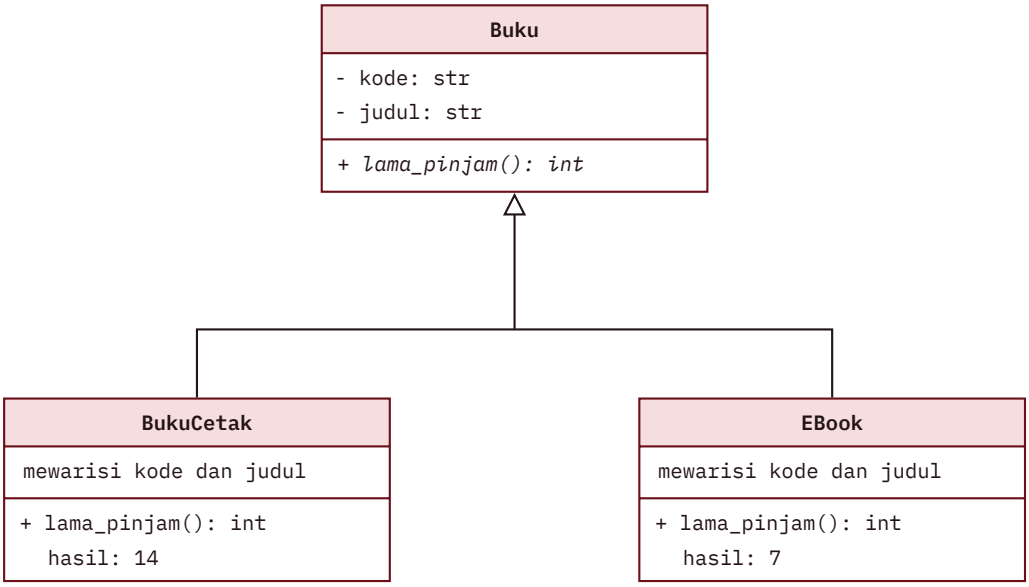
11.4 Class Relationship

Class relationship menjelaskan bagaimana object dari suatu class berhubungan dengan object class lain. Pemilihannya didasarkan pada kebutuhan dan tanggung jawab, bukan pada keinginan memakai semua jenis relasi sekaligus. Perbedaan hubungan *is-a* dan *has-a* dibahas oleh [Lutz, 2025]; pertimbangan inheritance dan composition juga dibahas oleh [North, 2025] pada Bab 5.

Inheritance untuk Jenis Buku

`BukuCetak` dan `EBook` sama-sama merupakan jenis `Buku`. Keduanya memiliki kode serta judul, tetapi memberikan hasil berbeda untuk `lama_pinjam()`. Karena itu, hubungan ini sesuai dimodelkan dengan inheritance.

Class `Buku` direncanakan sebagai abstract class yang mendefinisikan kontrak `lama_pinjam()`. Layanan `Perpustakaan` cukup memanggil operasi tersebut pada object buku yang dipilih. Layanan tidak perlu menyalin percabangan "jika cetak maka 14, jika e-book maka 7" di setiap proses. Pemanggilan dengan antarmuka yang sama dan hasil sesuai jenis object merupakan penerapan polymorphism.



Gambar 11.2. Generalisasi jenis buku. Segitiga kosong mengarah ke class yang lebih umum; nama class dan operasi abstract ditulis miring.

Association pada Catatan Peminjaman

`Peminjaman` berasosiasi dengan `Anggota` dan `Buku`. Catatan itu perlu mengetahui siapa peminjamnya dan buku apa yang dipinjam. Sebuah peminjaman bukan jenis anggota atau jenis buku, sehingga kedua hubungan tersebut tidak dibuat dengan inheritance.

Sebaliknya, anggota dan buku tidak harus menyimpan salinan riwayat masing-masing. Pada rancangan ini, layanan perpustakaan memperoleh riwayat dari koleksi `Peminjaman` berdasarkan ID atau kode yang diperlukan. Keputusan ini mengurangi tempat yang harus diperbarui saat transaksi berlangsung.

Aggregation dan Composition dalam Lingkup Model

Daftar buku dan anggota pada `Perpustakaan` dimodelkan sebagai *aggregation*: layanan menerima object yang didaftarkan dan mengelolanya sebagai kumpulan. Buku dan anggota tetap mempunyai identitas sendiri di luar satu transaksi peminjaman.

Koleksi `Peminjaman` dimodelkan sebagai *composition* dalam rancangan ini: setiap catatan dibuat oleh satu perpustakaan dan menjadi bagian riwayat perpustakaan tersebut. Catatan tidak dipakai sebagai milik bersama oleh dua perpustakaan. Kepemilikan ini adalah keputusan model untuk proyek satu perpustakaan, bukan kesimpulan dari letak baris pembuatan object di dalam constructor.

Menutup aplikasi tidak berarti menghapus riwayat. Object dalam memori dapat berakhir, tetapi keadaan perpustakaan disimpan ke file dan dibangun kembali saat dibuka. Siklus hidup data domain perlu dibedakan dari umur sebuah variabel atau proses Python.

Multiplicity dan Batas Peminjaman Aktif

Multiplicity menunjukkan banyaknya object yang dapat terhubung pada ujung suatu relasi. Gunakan `1` untuk tepat satu, `0..1` untuk nol atau satu, dan `0..*` untuk nol atau lebih.

Tabel 11.10. Multiplicity pada rancangan perpustakaan.

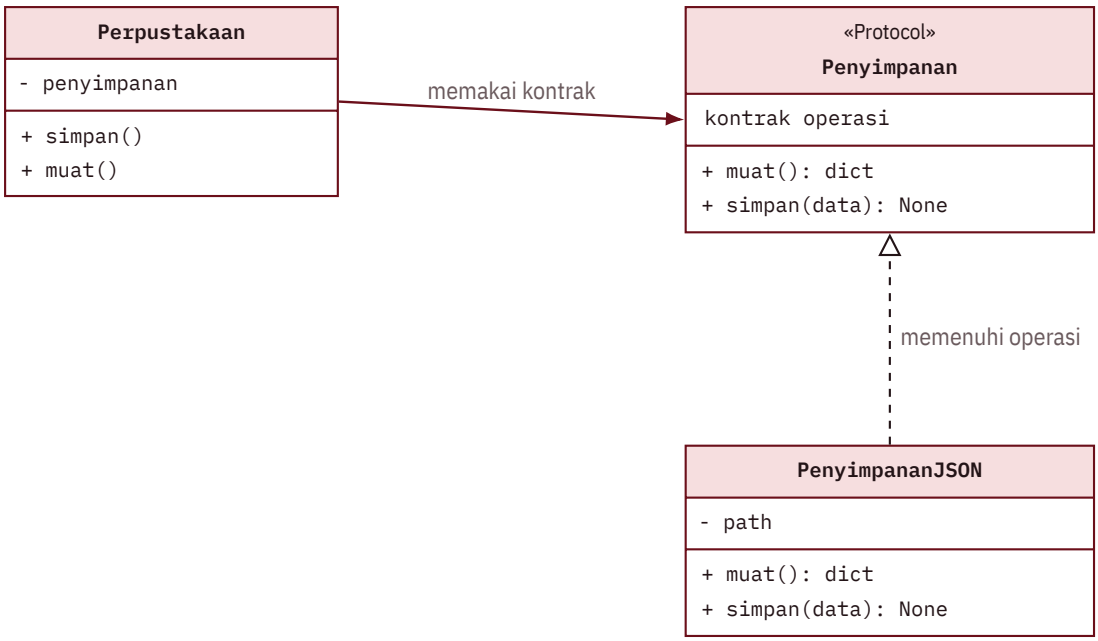
Dilihat dari	Jumlah	Makna
Satu perpustakaan ke buku	`0..*`	Perpustakaan dapat belum memiliki buku atau mempunyai banyak buku terdaftar.
Satu perpustakaan ke anggota	`0..*`	Perpustakaan dapat memiliki banyak anggota.
Satu perpustakaan ke peminjaman	`0..*`	Seluruh catatan peminjaman menjadi bagian riwayatnya.
Satu peminjaman ke anggota	`1`	Setiap catatan mempunyai tepat satu peminjam.
Satu peminjaman ke buku	`1`	Setiap catatan mencatat tepat satu kode buku.
Satu buku ke riwayat peminjaman	`0..*`	Buku dapat dipinjam berkali-kali pada waktu berbeda.

Relasi buku ke *seluruh riwayat* memakai `0..*`, bukan `0..1`. Batas paling banyak satu hanya berlaku pada catatan yang masih *aktif*. Karena itu, tambahkan aturan tertulis: *un-*

tuk setiap kode buku, jumlah peminjaman aktif tidak boleh lebih dari satu. Angka pada garis relasi saja belum menyatakan batas berdasarkan keadaan transaksi.

Hubungan Model, Penyimpanan, dan Tampilan

`Perpustakaan` memakai kontrak `Penyimpanan`, sedangkan `PenyimpananJSON` menyediakan operasinya. Karena object penyimpanan disimpan dan digunakan oleh layanan, hubungan layanan ke kontrak dapat digambar sebagai association. Implementasi yang memenuhi kontrak digambar dengan garis realisasi putus-putus menuju kontrak.



Gambar 11.3. Kontrak penyimpanan memisahkan layanan dari format file. Pada Python, implementasi dapat memenuhi `Protocol` secara struktural tanpa mewarisinya secara eksplisit.

Tampilan pada Bab 13 memanggil layanan yang sama untuk mendaftarkan, meminjamkan, dan mengembalikan buku. Model tidak perlu mengimpor `tkinter`. Mengganti tata letak tombol tidak mengubah aturan peminjaman; mengganti penyimpanan pun tidak menuntut pengubahan seluruh event handler.

11.5 Class Diagram Sederhana

Class diagram menggambarkan struktur class dan hubungan di antaranya. Diagram pada Gambar 11.4 memakai notasi UML yang disederhanakan untuk menunjukkan keputusan yang akan diimplementasikan. Diagram ini dibaca bersama kebutuhan dan kontrak, bukan sebagai pengganti keduanya.

Membaca Kotak dan Garis Relasi

Kotak class terdiri atas tiga bagian: nama class, attribute, dan operasi. Penulisan `nama: tipe` menunjukkan data yang disimpan, sedangkan `method(...)` menunjukkan operasi

yang dapat dipanggil. Tanda + menandai antarmuka publik dan - menandai detail internal pada rancangan.

Tanda - pada diagram tidak menambahkan pembatasan akses secara otomatis di Python. Implementasinya tetap memakai konvensi penamaan dan property yang dipelajari pada Bab 2. Misalnya, kode disimpan sebagai data internal dan dibaca melalui property hanya-baca.

Tabel 11.11. Notasi hubungan yang digunakan pada diagram.

Relasi	Notasi	Cara membaca
Association	Garis utuh	Object saling terhubung; angka di ujung menyatakan multiplicity.
Aggregation	Belah ketupat kosong	Ujung belah ketupat berada pada class pengelola kumpulan.
Composition	Belah ketupat terisi	Ujung belah ketupat berada pada pemilik bagian.
Inheritance	Segitiga kosong	Segitiga menunjuk class yang lebih umum.
Realisasi kontrak	Garis putus-putus dan segitiga kosong	Segitiga menunjuk kontrak yang dipenuhi implementasi.

Diagram Class Domain

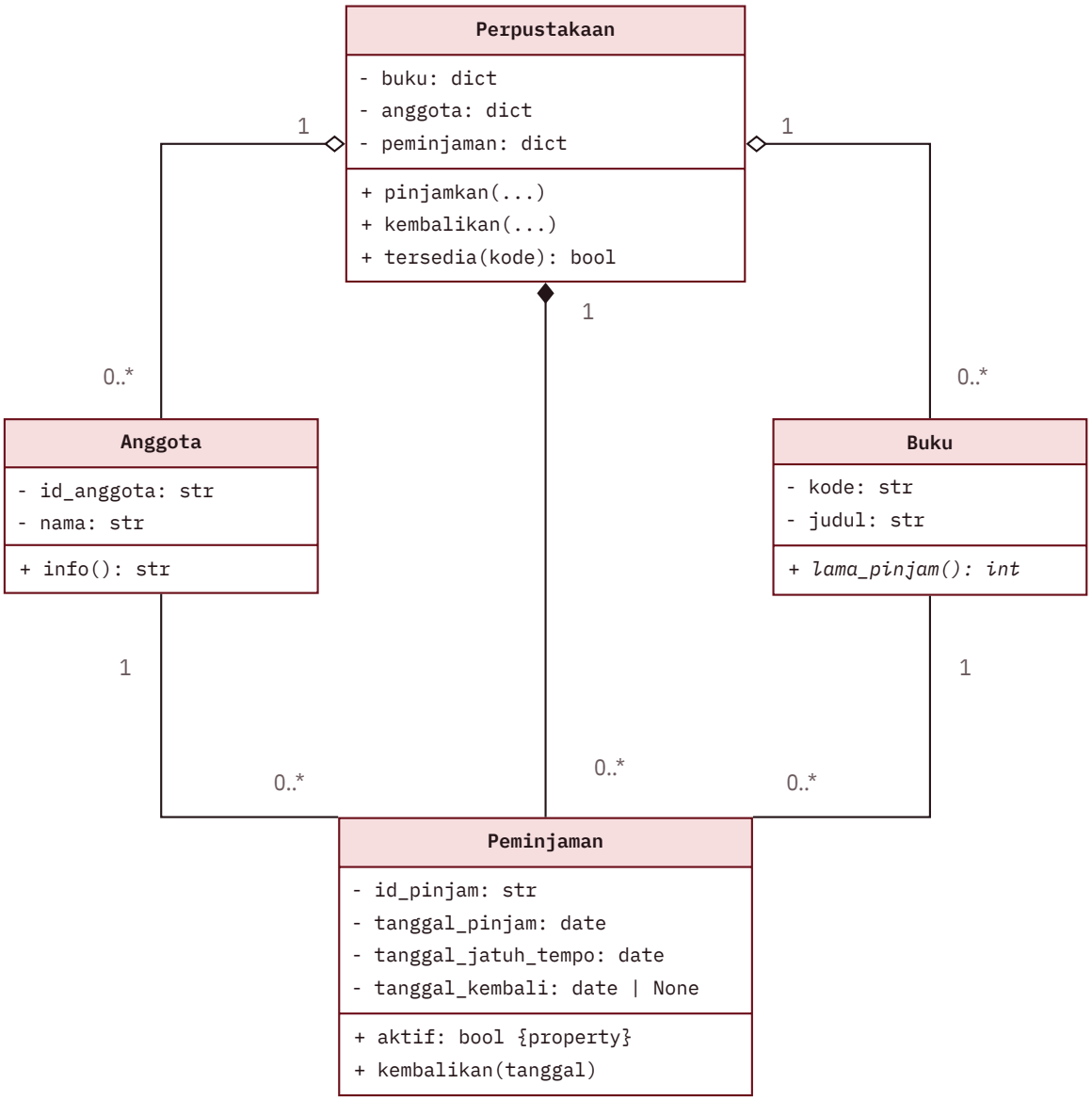
Diagram berikut menampilkan empat class utama. Turunan `BukuCetak` dan `EBook` sudah dirinci pada Gambar 11.2, sedangkan komponen file berada pada Gambar 11.3. Pemisahan gambar menjaga agar nama, attribute, dan garis relasi tetap terbaca.

Pada `Peminjaman`, rujukan `anggota` dan `buku` ditampilkan sebagai garis association sehingga tidak diulang sebagai daftar attribute pada kotaknya. Property `aktif` merupakan informasi turunan yang hanya dibaca. Nama `info()` pada `Anggota` mewakili operasi penyajian teks data anggota; operasi tersebut tidak membuka jendela atau menampilkan kotak pesan.

Menelusuri Diagram melalui Satu Transaksi

Gunakan kembali object M001, B001, dan P001 untuk memeriksa apakah diagram mendukung kebutuhan.

1. Layanan mencari M001 pada dictionary anggota dan B001 pada dictionary buku. Jika salah satunya tidak ditemukan, transaksi ditolak.
2. Layanan memeriksa catatan peminjaman. Jika ada catatan aktif untuk B001, transaksi ditolak tanpa mengubah koleksi.



Gambar 11.4. Class diagram inti perpustakaan. Satu catatan peminjaman terhubung dengan satu anggota dan satu buku; satu buku dapat memiliki banyak catatan sepanjang riwayatnya. Operasi tambahan dijelaskan dalam kontrak method.

3. Layanan memanggil `lama_pinjam()` pada B001. Object `BukuCetak` mengembalikan 14 hari.
4. Layanan membuat P001, menghubungkannya ke M001 dan B001, lalu menyimpan tanggal pinjam serta jatuh temponya.
5. Saat P001 dikembalikan, layanan meminta catatan itu memeriksa tanggal kembali dan menyelesaikan transaksi.
6. Ketersediaan B001 dihitung kembali. Karena P001 sudah tidak aktif, B001 dapat dipinjam lagi dan P001 tetap berada dalam riwayat.

Tabel 11.12. Perubahan keadaan yang diharapkan dari skenario rancangan.

Keadaan	Peminjaman aktif B001	Dampak
Sebelum dipinjam	Tidak ada	B001 tersedia.
P001 dibuat	P001	B001 tidak tersedia untuk peminjaman lain.
Percobaan pinjam kedua	Tetap P001	Permintaan ditolak; tidak ada catatan baru.
P001 dikembalikan	Tidak ada	B001 tersedia; riwayat P001 tetap tersimpan.

Tabel tersebut adalah hasil yang harus dibuktikan saat implementasi, bukan laporan program yang sudah dijalankan. Jika sebuah keadaan tidak dapat dijelaskan melalui class dan method yang dirancang, perbaiki pembagian tanggung jawab sebelum menulis aplikasinya.

Memeriksa Rancangan sebelum Implementasi

Periksa paling tidak enam keadaan: pendaftaran kode ganda, peminjaman sah, peminjaman buku yang masih aktif, peminjaman oleh anggota yang tidak terdaftar, pengembalian sah, dan pengembalian kedua untuk catatan yang sudah selesai. Tambahkan pemeriksaan simpan–muat untuk memastikan relasi tidak berubah setelah aplikasi dibuka ulang.

Pemeriksaan sejak tahap rancangan membantu menemukan kekurangan sebelum banyak kode bergantung pada keputusan tersebut. Wilson mencontohkan pengembangan melalui komponen kecil dan pemeriksaan perilaku yang dapat diulang [Wilson, 2024]. Bentuk latihan dengan syarat yang jelas dan hasil yang dapat diperiksa juga digunakan dalam kumpulan latihan [Edcorner Learning, tt].

Dokumen yang Dibawa ke Bab 12

1. Daftar kebutuhan F01–F06, kebutuhan kualitas, batas aplikasi, dan aturan bisnis.
2. Contoh object beserta kode, ID, dan keadaan awal yang dipakai untuk pemeriksaan.
3. Daftar class, attribute, method, serta kontrak operasi utamanya.
4. Diagram relasi dengan multiplicity dan catatan batas satu peminjaman aktif per buku.
5. Rancangan JSON dan aturan membangun kembali hubungan melalui ID.
6. Skenario berhasil dan gagal, lengkap dengan perubahan keadaan yang diharapkan.

Bab 12 akan mengubah dokumen tersebut menjadi module model dan penyimpanan yang dapat diperiksa dari terminal. Bab 13 kemudian menambahkan GUI yang memanggil model itu. Jika suatu kontrak perlu diubah saat implementasi, catat alasannya dan perbarui kebutuhan atau diagram yang berkaitan.

11.6 Rangkuman

- Analisis kebutuhan menjelaskan pengguna, ruang lingkup, fungsi, kualitas, aturan, dan bukti keberhasilan aplikasi.
- Kebutuhan fungsional menyatakan pekerjaan aplikasi; kebutuhan nonfungsional menyatakan kualitas atau batas kerjanya.
- Identifikasi object dimulai dari contoh konkret. Kata benda dapat menjadi kandidat object, tetapi sebagian cukup menjadi attribute, aktor, atau komponen tampilan.
- Class mengelompokkan object sejenis. Tanggung jawabnya menentukan attribute yang diperlukan dan method yang disediakan.
- Kontrak method menjelaskan masukan, hasil, syarat, serta kegagalan. Aturan lintas-object dijaga oleh layanan aplikasi.
- Data turunan seperti status aktif tidak perlu disimpan sebagai flag kedua. Data historis seperti tanggal jatuh tempo perlu mempertahankan keputusan saat transaksi dibuat.
- Inheritance digunakan untuk hubungan jenis buku; association menghubungkan peminjaman dengan anggota dan buku. Aggregation dan composition dipilih sesuai kepemilikan dalam model.
- Multiplicity pada riwayat buku adalah `0..*`; batas satu peminjaman aktif dituliskan sebagai aturan tambahan.
- Class diagram, kontrak, skenario, dan rancangan penyimpanan harus menggambarkan keputusan yang sama sebelum aplikasi diimplementasikan.

11.7 Kuis Teoritis

Pilihan Ganda

Pilihlah satu jawaban yang paling tepat berdasarkan ruang lingkup studi kasus dalam bab ini.

1. Pernyataan yang termasuk kebutuhan fungsional adalah ...

   a. model dapat diperiksa tanpa membuka GUI
   b. aplikasi menolak peminjaman buku yang masih dipinjam
   c. file menggunakan UTF-8
   d. pesan kesalahan mudah dipahami

2. Dalam rancangan awal, "judul buku" paling tepat menjadi ...

   a. aktor
   b. class layanan
   c. attribute pada buku
   d. method penyimpanan

3. B001 dan B002 memiliki judul yang sama tetapi kode berbeda. Keduanya merupakan ...

   a. dua object buku yang berbeda
   b. dua nama untuk object yang pasti sama
   c. satu class dan satu method
   d. data duplikat yang pasti harus dihapus

4. Multiplicity dari satu catatan peminjaman ke buku dalam studi kasus ini adalah ...

   a. `0..*`
   b. `0..1`
   c. `1..*`
   d. `1`

5. Hubungan antara `Peminjaman` dan `Anggota` paling tepat dinyatakan sebagai ...

   a. inheritance karena peminjaman adalah anggota
   b. association karena peminjaman merujuk peminjamnya
   c. inheritance karena kedua class memiliki ID
   d. hubungan yang tidak perlu dicatat

6. Pemeriksaan bahwa sebuah buku belum mempunyai peminjaman aktif terutama menjadi tanggung jawab ...

   a. layanan `Perpustakaan` yang melihat koleksi transaksi

b. warna tombol Pinjam
c. module `json` tanpa bantuan model
d. class `Anggota` tanpa melihat buku atau transaksi lain

7. Menghitung ketersediaan dari peminjaman aktif membantu ...

a. menghapus semua riwayat saat pengembalian
b. menghindari kebutuhan identitas buku
c. menghindari dua sumber status yang dapat berbeda
d. menjamin semua kebutuhan aplikasi otomatis terpenuhi

8. Bukti bahwa model terpisah dari GUI adalah ...

a. semua class berada di dalam event handler
b. setiap method model membuka kotak pesan
c. aturan hanya diperiksa ketika tombol ditekan
d. operasi peminjaman dapat diperiksa dari terminal tanpa membuka jendela

9. Cara menyimpan relasi peminjaman dalam JSON adalah ...

a. menyimpan hasil `id(object)` sebagai identitas tetap
b. menyimpan ID anggota dan kode buku, lalu menghubungkannya kembali saat dimuat
c. menyimpan gambar tampilan GUI saja
d. menyimpan object Python langsung dengan `json.dump()` tanpa konversi

10. Segitiga kosong pada hubungan `BukuCetak` dengan `Buku` menunjuk ke ...

a. object yang dibuat paling akhir
b. class turunan `BukuCetak`
c. class yang lebih umum, yaitu `Buku`
d. file JSON

Uraian

1. Uraikan permintaan “buat aplikasi perpustakaan yang mudah dipakai” menjadi dua kebutuhan fungsional dan dua kebutuhan kualitas yang dapat diperiksa.
2. Jelaskan mengapa petugas, anggota, dan nama anggota tidak otomatis menjadi tiga class yang setara dalam ruang lingkup proyek ini.
3. Mengapa riwayat buku memakai multiplicity `0..*`, padahal sebuah buku hanya boleh memiliki satu peminjaman aktif? Berikan contoh dua transaksi pada waktu berbeda.
4. Jelaskan alasan tanggal jatuh tempo disimpan saat transaksi dibuat, sedangkan status aktif dan ketersediaan diturunkan dari data yang sudah ada.
5. Jelaskan langkah memuat P001 dari JSON agar rujukan anggota dan bukunya menunjuk object terdaftar yang benar, serta dua keadaan data yang harus ditolak.

CEK JAWABAN PILIHAN GANDA

1 b; 2 c; 3 a; 4 d; 5 b; 6 a; 7 c; 8 d; 9 b; 10 c. Cocokkan alasan jawaban Anda dengan kebutuhan, kontrak, dan diagram, terutama jika jawaban benar diperoleh dengan menebak.

11.8 Praktik Mandiri

Hasil praktik pada bab ini berupa dokumen rancangan. Tuliskan keputusan dan alasannya agar dapat dipakai sebagai acuan implementasi, baik oleh Anda sendiri maupun teman satu kelompok.

Praktik 1: Melengkapi Dokumen Kebutuhan

Gunakan studi kasus perpustakaan dari bab ini. Pilih dua kebutuhan dari F01–F06, kemudian lengkapi masing-masing dengan aktor, kondisi awal, masukan, langkah utama, hasil berhasil, dan dua alur penolakan. Salah satu kebutuhan yang dipilih harus berupa peminjaman atau pengembalian.

Untuk setiap alur penolakan, sebutkan data mana yang harus tetap sama. Misalnya, peminjaman kedua yang ditolak tidak boleh menambah riwayat atau mengubah peminjam pada catatan yang masih aktif.

Praktik 2: Menilai Dampak Perubahan Aturan

Perpustakaan ingin menambahkan aturan baru: satu anggota paling banyak mempunyai tiga peminjaman aktif. Aturan ini merupakan pengembangan latihan, belum menjadi batas pada rancangan dasar bab ini.

Langkah Pengerjaan

1. Tambahkan kebutuhan dan kriteria penerimaan untuk batas tiga peminjaman aktif.
2. Tentukan class serta method yang perlu memeriksa aturan itu, dan jelaskan data apa yang diperlukannya.
3. Jelaskan apakah aturan tersebut membutuhkan class baru. Berikan alasan, bukan hanya menambah class agar diagram tampak lebih lengkap.
4. Buat skenario peminjaman ketiga yang berhasil, peminjaman keempat yang ditolak, dan peminjaman baru setelah satu buku dikembalikan.
5. Perbarui kontrak atau catatan pada diagram yang terdampak. Tunjukkan bagian rancangan yang tetap dapat digunakan.

Praktik 3: Merancang Peminjaman Alat Laboratorium

Rancang aplikasi peminjaman alat laboratorium dengan petugas sebagai pengguna. Setiap alat memiliki kode dan nama; mahasiswa terdaftar dapat meminjam alat yang tersedia; pengembalian mempertahankan riwayat. Tentukan sendiri dua jenis alat dengan lama pinjam berbeda dan jelaskan alasan perbedaannya.

- Tuliskan ruang lingkup, minimal lima kebutuhan fungsional, tiga kebutuhan kualitas, dan aturan bisnisnya.
- Identifikasi minimal dua contoh object untuk setiap class domain yang direncanakan.
- Susun daftar class, attribute, method, serta kontrak peminjaman dan pengembalian.
- Gambarkan class diagram dengan nama relasi dan multiplicity yang jelas. Gunakan inheritance hanya jika hubungan *is-a* dapat dijelaskan.
- Rancang data JSON yang mempertahankan hubungan antar-object melalui identitas.
- Siapkan minimal enam skenario pemeriksaan, termasuk kegagalan dan simpan–muat data.

KRITERIA PENILAIAN

- **Kebutuhan dan aturan, 25%**: jelas, dapat diperiksa, dan sesuai ruang lingkup.
- **Identifikasi object, 20%**: identitas dan keadaan konkret dibedakan dari class serta attribute.
- **Pembagian tanggung jawab, 20%**: attribute dan method mendukung kebutuhan tanpa menumpuk semua pekerjaan pada satu class.
- **Relasi dan diagram, 20%**: arah generalisasi, kepemilikan, serta multiplicity sesuai dengan penjelasan.
- **Pemeriksaan rancangan, 15%**: skenario berhasil, gagal, dan pemuatan data menghasilkan keadaan yang konsisten.

Daftar Pustaka

Sumber yang dirujuk dalam bab ini.

[Anonim, tt] Anonim (t.t.). Object-oriented programming with python. Penulis, penerbit, dan tahun terbit tidak tercantum pada salinan digital yang digunakan.

[Edcorner Learning, tt] Edcorner Learning (t.t.). *Python Object Oriented Programming Exercises: Become a Pro Developer*. Penerbit tidak tercantum. Tahun terbit tidak tercantum pada salinan digital yang digunakan.

[Lutz, 2025] Lutz, M. (2025). *Learning Python: Powerful Object-Oriented Programming*. O'Reilly Media, Sebastopol, CA, 6 edition.

[North, 2025] North, M. (2025). *Object-Oriented Programming in Python: Design and Build Programs Using Classes, Objects, and the Four Pillars of OOP*, volume 1 of *Think, Code, Create – Advanced Python Programming Series*. Northward House Publishing, Brisbane, Queensland, Australia.

[Wilson, 2024] Wilson, G. (2024). *Software Design by Example: A Tool-Based Introduction with Python*. CRC Press.